PORTFOLIO 2026

포트폴리오

게임 클라이언트 프로그래머 | 김창홍

시스템 구조와 실행 흐름을 기반으로 문제를 분석하고, 설계를 통해 해결하는 개발자입니다.

010-4720-0578 | 1138aiie@gmail.com | github.com/CHGangRiO

지원 분야와 프로젝트 요약

게임 클라이언트 프로그래머 (신입)

구조 설계

입력/판단/행동을 계층으로 나누고 공통 로직을 상위로 올려, 캐릭터 종류가 늘어도 실행 흐름을 재사용하도록 설계합니다.

원인 추적

증상이 아니라 호출 순서와 데이터 흐름을 따라가 원인을 특정하고, 확인 방법을 남겨 재현 가능하게 만듭니다.

선택과 검증

대안을 비교해 제약 조건에 맞는 방식을 고르고, 같은 조건에서 전후를 측정해 결과를 확인합니다.

01 **Unity 6 | 3D 액션 게임 (1인)**
2026.03 ~ 2026.03
플레이어 전투, 몬스터 AI, 내비게이션 이동 P.03

02 **DirectX11 | 3D 액션 게임 (1인)**
2025.02 ~ 2025.04
모델 로드, 스켈레탈 애니메이션, 전투, 카메라 P.11

03 **DirectX11 | 3D 액션 게임 (5인)**
2025.04 ~ 2025.06
몬스터 AI 및 행동 제어 (몬스터 담당) P.18

PROJECT 01

Unity 6을 이용한 3D 액션 게임

1인 제작 | 2026.03.02 ~ 2026.03.27 (4주)

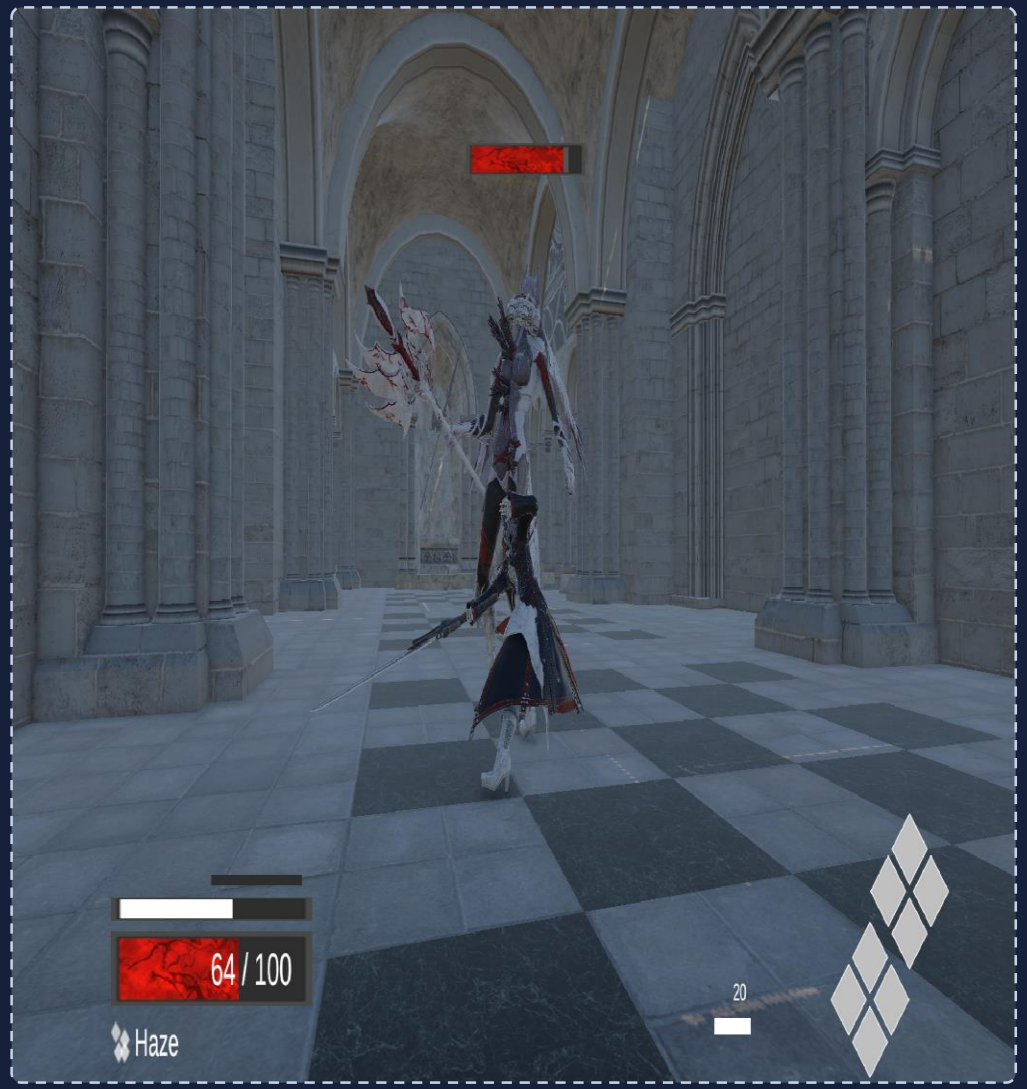
담당 플레이어 전투 시스템, 몬스터 AI, 내비게이션 기반 이동 구현

기반 Unity 6 위에서 게임플레이 코드를 전부 직접 작성했습니다. 외부 에셋은 모델과 애니메이션에 한정합니다.

환경 Unity 6, C#, URP, Input System, UGUI, Cinemachine

다루는 것 입력/판단/행동 분리 구조, 데이터 기반 공격 구성, 애니메이션 이벤트와 상태 전이의 순서, NavMesh 이동

저장소 github.com/CHGangRiO/p-11u-mg-ps



입력 / 판단 / 행동 분리 기반 행동 제어 구조

구조

Player Layer

PlayerInput
Input System 콜백으로 입력만 저장

↓

PlayerBrain
CharacterCommand 생성, 락온 상태 반영

↓

PlayerStateMachine
현재 상태 Tick, 애니메이션 이벤트 중계

↓

상태 구현

Idle	Move	Attack
Dodge	Hit	Dead

Enemy Layer

EnemyPerception
시야 및 거리 기반 타겟 탐색 Scan()

↓

EnemyBrain
Blackboard 갱신, 타겟팅 및 공격 가능 판정

↓

EnemyStateMachine
현재 상태 Tick, 애니메이션 이벤트 중계

↓

상태 구현

Idle	Chase	Attack
Dodge	Hit	Dead

Engine Layer - Player / Enemy 공통

Resources Health / Stamina / Mana	**Stats** CharacterStats / StatModifier	**Combat** HitBox / HurtBox / DamageReceiver	**Animation** AnimationController / EventRelay
Movement IMovementController / RootMotion	**Targeting** LockOnController / TargetDetector	**StateMachine** IState	**Common** CharacterBase / GameConstants

프레임 처리 순서 Player BuildCommand() → Tick() → ConsumeFrameInput() | Enemy BuildDecisionContext() → Tick()

설계 의도

Player와 Enemy가 각자 입력을 받고 상태를 바꾸면 같은 형태의 코드를 두 번 쓰게 되고, 체력이나 스태미나 같은 값도 양쪽에 흩어집니다.

구성

입력원(PlayerInput/EnemyPerception)이 Brain에 판단 재료를 넘기고, 전이 판단은 각 상태가, 자원/스탯/피해 처리는 Engine 계층이 맡습니다. 두 캐릭터는 같은 계층 순서를 따르되 상태 클래스는 각자 가집니다.

효과

몬스터를 추가할 때 EnemyStateMachine과 Engine 계층을 그대로 쓰고 상태만 늘리면 됩니다. 자원 소모에 실패하면 상태 진입 자체가 중단되어 UI 값과 어긋나지 않습니다.

공격 한 번이 처리되는 흐름

구조

1 입력 → 명령
- **PlayerInput** — AttackNormalPressed 저장
- **PlayerBrain** — CharacterCommand 생성

2 상태 전이
- **PlayerStateMachine** — 상태 전이 발생
- **PlayerIdleState** — 현재 상태 종료
- **PlayerAttackState** — 공격 상태 진입

3 공격 데이터
- **WeaponHandler** — 무기 콤보 호출
- **WeaponData** — 무기 콤보 배열 반환
- **AttackDefinition** — 애니메이션 이름 반환

4 자원 / 재생
- **Stamina** — 스태미나 사용 시도
- **CharacterAnimationController** — 애니메이션 전환

5 판정 구간
- **AnimationEventRelay** — 공격 예약 활성화/비활성화
- **HitBox** — 충돌 시작/종료

6 피해 처리
- **HurtBox** — OnTriggerEnter 수신
- **DamageReceiver** — 공격측에서 데미지 구조체 전달
- **DamageCalculator** — 방어와 치명타 계산하여 피해량 적용

설계 의도

공격 판정 구간을 코드에서 시간으로 계산하면 애니메이션을 교체할 때마다 값을 다시 맞춰야 합니다. 판정 시점을 애니메이션이 직접 알려주게 했습니다.

구성

상태는 WeaponHandler에서 현재 무기의 콤보 배열을 받아 애니메이션 이름을 꺼내 재생하고, HitBox 활성/비활성은 AnimationEventRelay가 알립니다. 피해 계산은 DamageReceiver와 DamageCalculator가 맡습니다.

효과

애니메이션을 바꿔도 이벤트 위치만 조정하면 되고, 무기를 추가할 때 상태 코드를 고치지 않습니다. 콤보 연결도 같은 이벤트 구간에서 예약 처리합니다.

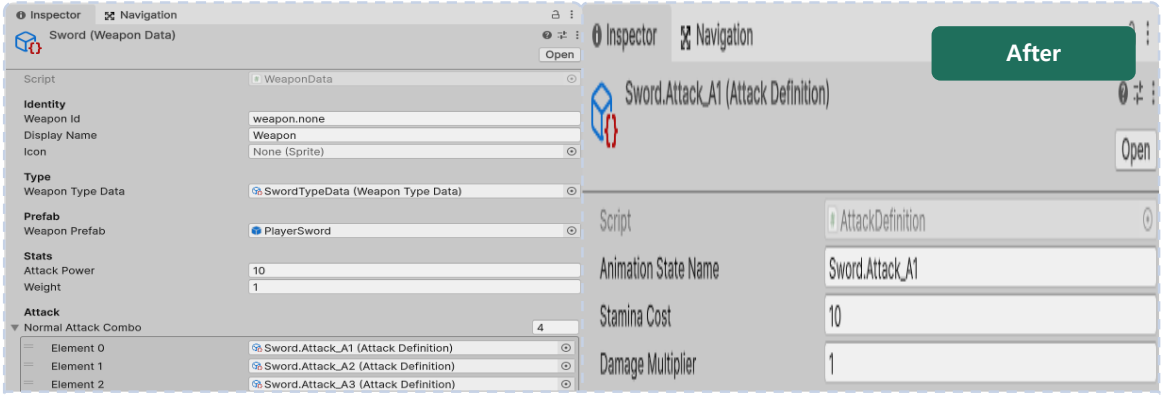
무기를 추가할 때마다 상태 코드를 고쳐야 했던 문제

문제 해결

Before

```
public class PlayerAttackState : PlayerBaseState
    , IAnimationEventListener
{
    private string[] _combo =
        { "Sword.Attack_A1", "Sword.Attack_A2" , "Sword.Attack_A3" };
}
```

After




```
public class PlayerAttackState : PlayerBaseState, IAnimationEventListener
{
    private AttackDefinition[] _combo;
```

문제

무기를 추가하거나 콤보를 바꿀 때마다 상태 클래스 코드를 직접 고쳐야 했습니다.

원인

공격 횟수와 애니메이션 이름을 상태가 직접 들고 있어, 데이터여야 할 값이 코드에 섞여 있었습니다.

선택

상태 안에서 무기별로 분기하면 무기가 늘수록 조건문이 늘고, 무기마다 상태 클래스를 상속하면 클래스가 늘어납니다. 값을 데이터로 빼는 쪽을 택했습니다.

구현

공격 구성을 WeaponData와 AttackDefinition으로 분리하고, 상태는 현재 무기 데이터를 참조해 실행만 담당하도록 바꿨습니다.

검증

서로 다른 콤보 구성의 무기 2종을 만들어, 코드 변경 없이 에셋 교체만으로 공격 구성이 바뀌는 것을 확인했습니다.

남은 한계

데이터 값이 잘못 들어가도 실행 시점에야 드러납니다. 에셋 단계에서 검사하는 처리는 넣지 못했습니다.

회피 도중 다시 공격 상태로 돌아가던 문제

문제 해결

Before



After



문제
몬스터가 공격 상태에서 회피 상태로 전이한 뒤, 회피 진행 도중 다시 공격 상태로 전이했습니다.

원인
CrossFade로 전이할 때 이전 애니메이션이 보간 구간에 남아 있었고, 그 구간에서 이전 클립의 이벤트가 호출되어 상태를 덮어썼습니다.

선택
전이 시 이벤트를 일괄 무시하면 정상 타이밍의 이벤트까지 사라집니다. 이벤트 발생 시점을 보간 구간 밖으로 옮기는 쪽을 택했습니다.

구현
이벤트 발생 프레임을 앞당겨, 상태 전이 이후에는 이전 클립의 이벤트가 호출되지 않도록 조정했습니다.

검증
상태 전이 시각과 이벤트 호출 시각을 함께 로그로 남겨, 전이 이후 이벤트가 오지 않는 것을 확인했습니다.

남은 한계
클립마다 이벤트 위치를 수동으로 조정해야 합니다. 전이 시 이전 클립의 이벤트 구독을 끊는 방식이 더 안전합니다.

목표 방향을 바라보기 전에 회피가 실행되던 문제

문제 해결

Before



After



문제
회피 입력이 들어오면 즉시 상태가 바뀌어, 캐릭터가 목표 방향을 바라보기 전에 회피가 실행됐습니다.

원인
입력을 받은 프레임에 바로 전이를 수행해, 회전 처리보다 상태 전이가 먼저 실행되는 순서 문제였습니다.

선택
회전 속도를 높여 맞추는 방법은 동작이 부자연스러워지므로, 전이 시점을 회전 완료 이후로 미루는 쪽을 택했습니다.

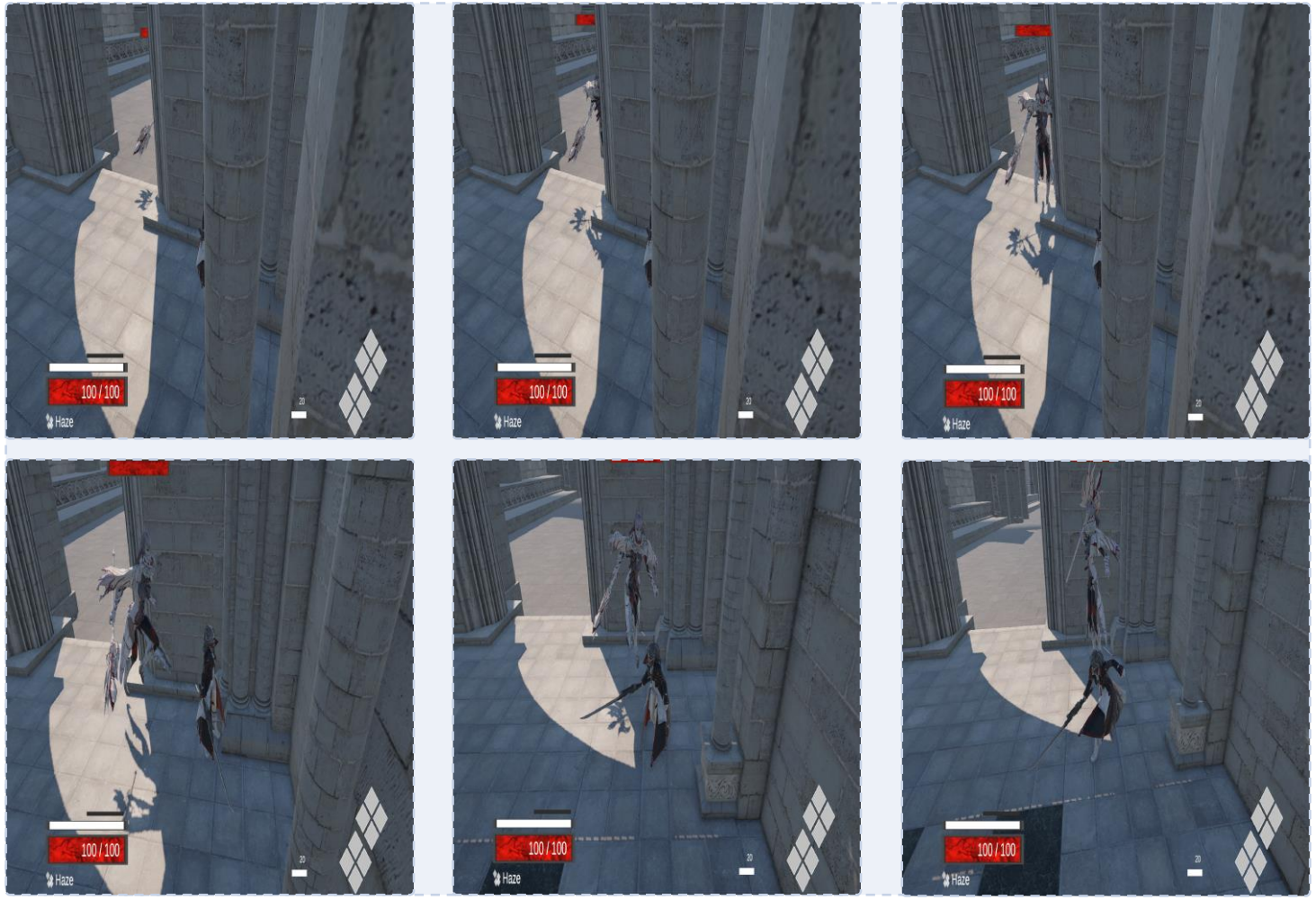
구현
회피 입력을 즉시 실행하지 않고 예약 플래그로 저장해 두었다가, 목표 방향으로 회전이 끝난 뒤 회피 상태로 전이하도록 바꿨습니다.

검증
여러 방향에서 회피를 반복해, 입력한 방향과 실제 회피 방향이 일치하는지 확인했습니다.

남은 한계
회전에 걸리는 시간만큼 회피 시작이 늦어집니다. 입력 반응성과 방향 정확도 사이의 균형은 더 조정할 여지가 있습니다.

몬스터 인식과 NavMesh 기반 이동

구조



설계 의도

직선 추적만 쓰면 벽이나 장애물에 걸려 몬스터가 제자리에서 밀립니다. 경로를 따라 이동하되 애니메이션의 이동감은 유지해야 했습니다.

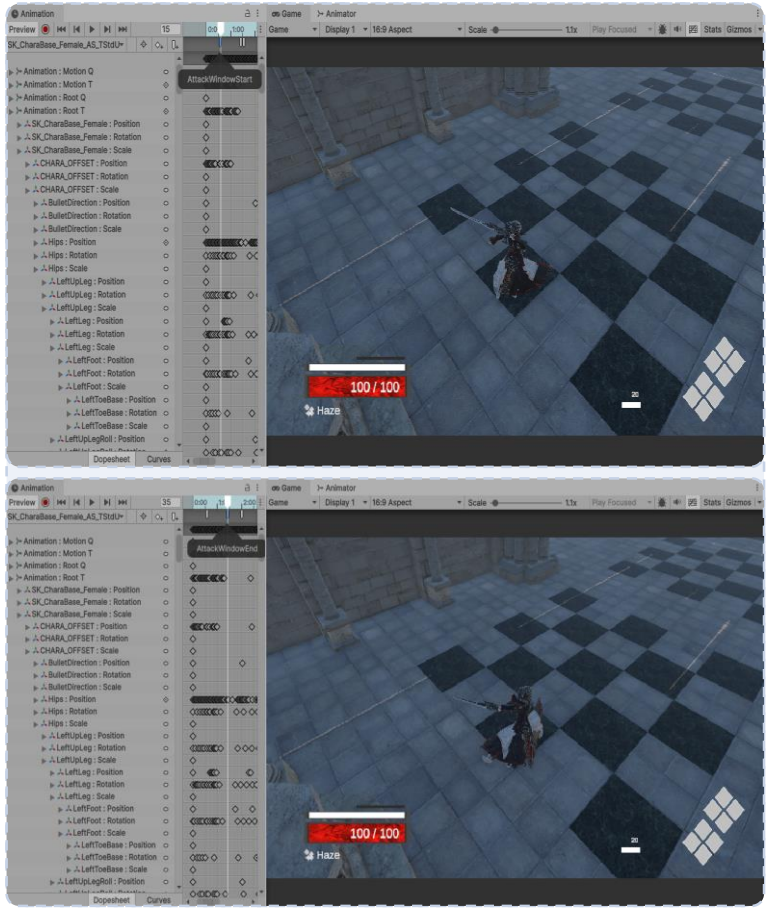
구성

Perception이 플레이어를 인식하면 Brain이 추적 의도를 만들고, 이동 상태가 NavMesh 경로를 따라갑니다. 도착 판정은 목적지까지의 거리로 처리했습니다.

효과

Player와 같은 실행 흐름을 그대로 쓰면서 입력원만 인식으로 바뀌므로, 몬스터를 추가해도 이동과 추적 로직을 다시 만들지 않습니다.

그 외 구현 — 전투 / 입력 / UI



애니메이션 이벤트 기반 판정

충돌 활성, 비활성 시점을 애니메이션 이벤트로 제어해, 모션과 판정 구간이 어긋나지 않게 했습니다.



HUD 연동

값 변경을 전달받아 갱신하도록 해, UI가 캐릭터 내부 값을 직접 참조하지 않습니다.



콤보 입력 버퍼

허용 구간에 들어온 입력을 예약해 두었다가 다음 타로 이어, 정확한 타이밍이 아니어도 콤보가 연결됩니다.

PROJECT 02

DirectX11을 이용한 3D 액션 게임

1인 제작 | 2025.02.03 ~ 2025.04.25

담당 모델 로드 및 스켈레탈 애니메이션, 전투, 카메라 구현

기반 DirectX 11 기반 자체 제작 엔진

환경 C++, DirectX11 (HLSL 5.0, FX11), DirectXTK, Assimp, IMGUI, FMOD

다루는 것 파츠로 분리된 모델의 스켈레톤 통합 / 계층형 상태 머신 기반 전투 / 로딩 시간 단축 / 애니메이션 이동 일치 / 공격 궤적

영상 youtu.be/BVLWR92e69k



파츠로 분리된 모델을 하나의 캐릭터로

구조

CModel_Pos

본 계층, 애니메이션, 루트 모션, 전환 보간을 담당
(메시 없고 렌더링하지 않음)

↓ 본 포인터를 얕은 복사로 전달

CModel_Draw - 파츠별 메시/머티리얼/ 본은 CModel_Pos 것을 참조

Part_Face CModel_Draw	**Part_Hair** CModel_Draw	**Part_Mask** CModel_Draw	**Part_Inner** CModel_Draw
Part_InnerFrame CModel_Draw	**Part_Drain** CModel_Draw	**Part_DrainMask** CModel_Draw	**Part_Sword** CModel_Draw

↓ 각 파트 오브젝트가 필요한 모델만 소유

CPlayer : IContainerObject - 파트 오브젝트 16개 (PARTOBJ)

Player_Anim CModel_Pos	**Player_Face** CModel_Draw	**Player_Hair** CModel_Draw	**Player_Mask** CModel_Draw
Player_Inner CModel_Draw	**Player_InnerFrame** CModel_Draw	**Player_Drain** CModel_Draw	**Player_DrainMask** CModel_Draw
Player_Sword CModel_Draw, 트레일	**Player_Body** CModel	**Player_Weapon / Effect** 충돌, 이펙트	**UI Bar** HP 표시

프레임 흐름 Player_Anim.Play_RootAnimation() → 본 행렬 확정 → 각 파츠의 본 행렬 전달 → Render()

설계 의도

캐릭터가 얼굴, 머리, 의상, 무기 등 파츠 파일로 나뉘어 있고 일부 본이 중복되어, 파츠마다 애니메이션을 따로 돌리면 동작이 어긋납니다.

구성

본과 애니메이션은 CModel_Pos 하나가 갖고, 파츠 8종은 CModel_Draw로 메시와 머티리얼만 가집니다. 모델을 읽을 때 기존 본 목록을 넘겨 병합하고, 파일 이름으로 된 불필요한 본은 최상위 본 확인 단계에서 걸렀습니다.

효과

본 1세트를 파츠 8종이 공유하므로 같은 애니메이션이 전체에 동일하게 적용되고, 파츠를 추가해도 애니메이션 처리를 다시 만들지 않습니다.

계층형 상태 머신 기반 전투 제어

구조



설계 의도

상태가 늘수록 상태 간 전이 조건이 급격히 늘어, 어떤 상태에서 어디로 갈 수 있는지 추적하기 어려웠습니다.

구성

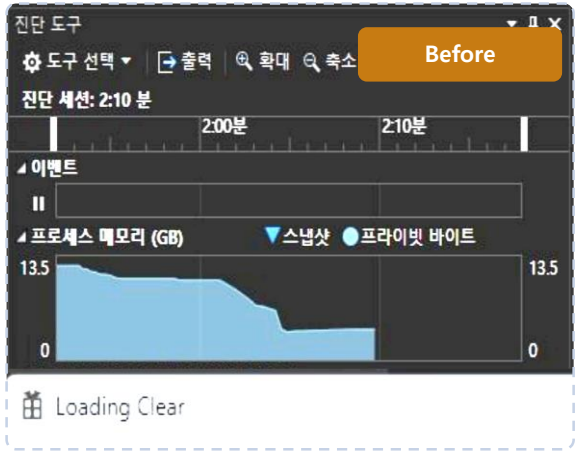
계층 간 전이는 FSMContainer가, 계층 안의 상태 전이는 StateMachine이, 애니메이션 지정과 종료 판정은 BaseState가 맡도록 3단으로 나눴습니다. Engine에 인터페이스를 두고 Client에서 플레이어용으로 구현했습니다.

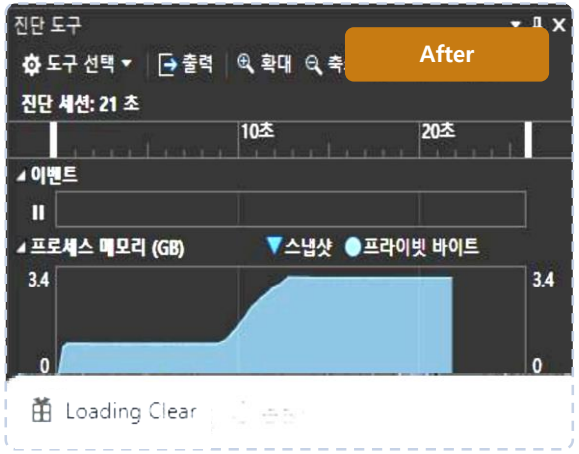
효과

피격/사망처럼 어느 계층에서나 발생하는 전이를 Container에서 한 번만 정의하면 되고, 상태를 추가할 때 영향 범위가 그 계층 안으로 제한됩니다.

모델 로딩에 실행마다 약 2분이 걸리던 문제

문제 해결





준비 단계 - 최초 1회 (Ver_Save)

원본 FBX	Assimp 전체 파싱	부분 직렬화	직렬화 자료 저장
*.fbx	fbx 전체 로드	본/메시 /애니메이션	필요한 데이터만 추출

실행 단계 - 매 실행 (Ver_Load)

*.bin 읽기	CModel_Pos	CModel_Draw	로딩 완료
역직렬화	애니메이션 bin 로드	파츠 bin 로드	파싱 단계 없음

문제

Assimp로 원본 모델을 직접 로드할 때 실행마다 약 2분이 걸렸고, 모델이 복잡할수록 초기 로딩이 크게 늘었습니다.

원인

원본 파일 전체를 파싱해야 했고, 애니메이션/본/메시 데이터를 매 실행 초기화 단계에서 다시 처리하고 있었습니다.

선택

비동기 로딩은 체감만 개선하고 반복 테스트 비용은 그대로, 모델 단순화는 재현도를 잃습니다. 준비 비용을 최초 1회로 옮기는 직렬화를 택했습니다.

구현

런타임에 필요한 메시/본/애니메이션만 추출해 별도 포맷으로 저장하고, 실행 시에는 역직렬화만 수행하도록 준비 단계와 실행 단계를 분리했습니다.

검증

동일 조건에서 반복 측정한 결과, 로딩이 약 2분에서 20초 이하로 줄었습니다. 반복 테스트와 디버깅 주기가 짧아졌습니다.

남은 한계

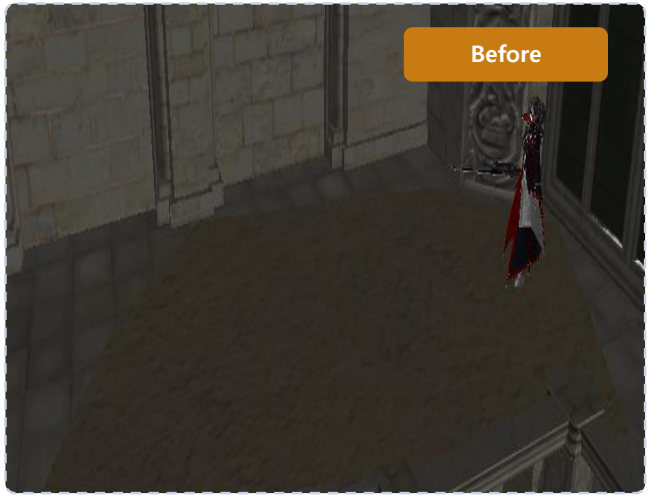
원본 모델이 바뀌면 다시 직렬화해야 하는데, 변경 감지와 자동 갱신까지는 넣지 못해 수동으로 처리했습니다.

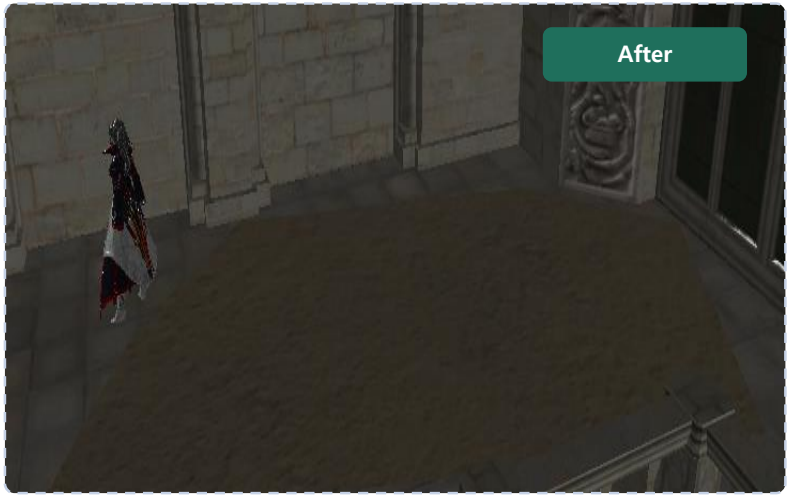
애니메이션 이동과 실제 월드 위치가 어긋나던 문제

문제 해결

루트 애니메이션







문제

대쉬처럼 이동이 포함된 애니메이션에서 화면상 캐릭터는 움직이는데 실제 월드 좌표는 제자리였습니다.

원인

이동량이 애니메이션의 로컬 좌표에만 적용되고 월드 행렬로 전달되지 않아, 연출 위치와 충돌 판정 위치가 어긋났습니다.

선택

애니메이션에서 이동을 빼고 코드로 이동시키면 모션과 속도가 어긋나므로, 애니메이션의 이동량을 그대로 월드로 옮기는 쪽을 택했습니다.

구현

이동을 담당하는 본의 로컬 위치를 저장해 두고, 다음 갱신에서 그 차이를 월드 행렬에 전달한 뒤 해당 본의 위치를 원점으로 되돌렸습니다.

검증

동일 시작 좌표에서 대쉬를 반복해 종료 좌표와 충돌 판정 위치가 일치하는지 확인했습니다.

남은 한계

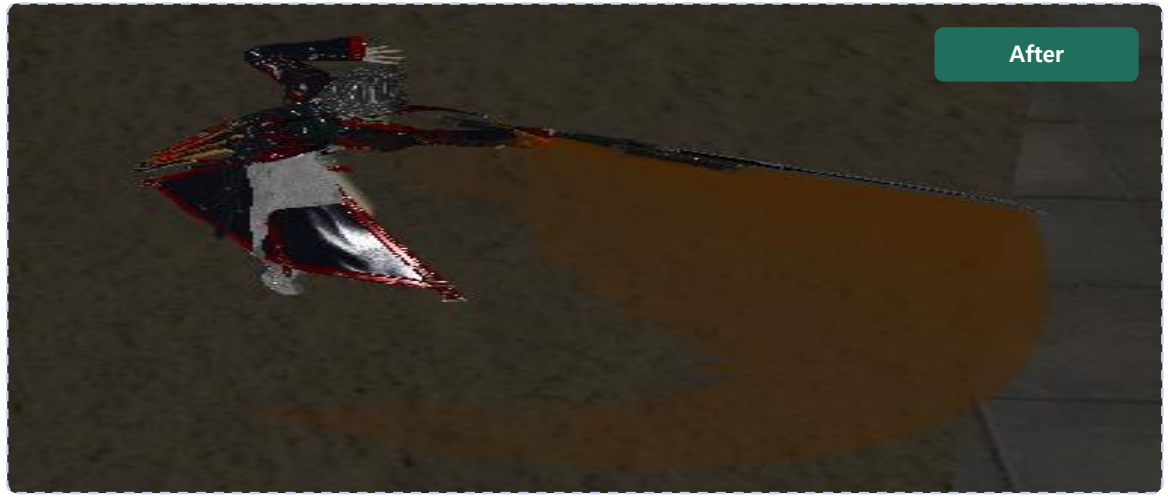
회전이 포함된 애니메이션에서는 회전량까지 함께 옮겨야 하는데, 이번 프로젝트에서는 이동량만 처리했습니다.

공격 궤적이 없어 타격감과 충돌 확인이 모두 어려웠던 문제

문제 해결

트레일 이펙트





문제

공격 시 무기 궤적이 남지 않아 타격감이 약했고, 충돌이 활성화된 구간을 직관적으로 확인할 수 없었습니다.

원인

현재 프레임의 무기 위치만 렌더링되어 이동 궤적이 남지 않았고, 충돌 여부는 피격 애니메이션으로만 간접 확인할 수 있었습니다.

선택

파티클로 흉내 내면 무기 위치와 어긋나므로, 실제 무기 좌표를 추적해 궤적 메시를 만드는 방식을 택했습니다.

구현

충돌이 활성화된 프레임에서 검 손잡이와 검 끝 좌표를 전달해 버퍼를 구성하고, 좌표 간격이 넓으면 중간 지점을 추가한 뒤 스플라인 보간으로 이어 메시를 생성했습니다.

검증

빠른 스윙 구간에서 궤적이 끊기지 않는지 확인했고, 충돌 활성 구간과 궤적 표시 구간이 일치하는지 함께 확인했습니다.

남은 한계

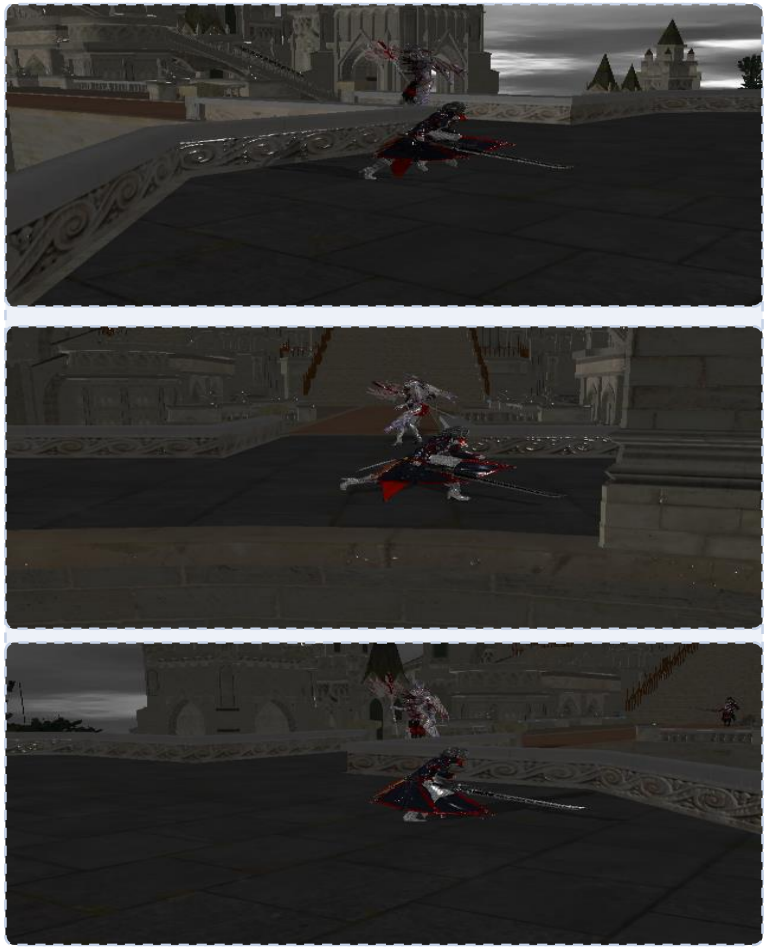
궤적 길이와 수명이 고정값이라 무기 종류별로 조정하려면 코드를 고쳐야 합니다.

그 외 구현 — 전투 / 카메라 / 이펙트



구면 좌표 카메라

방위각, 고각, 거리로 위치를 계산하고 고각에 상한을 두어 카메라가 뒤집히지 않게 했습니다.



카메라 락온

정면 15도 내 가장 가까운 대상을 타겟으로 두고, 타겟이 없으면 플레이어 정면으로 정렬합니다.



디졸브

사망 후 노이즈 텍스처 샘플링 값이 임계값보다 작은 픽셀을 버려 서서히 사라지게 했습니다.

PROJECT 03

DirectX11을 이용한 3D 액션 게임

5인 팀 제작 | 2025.04.28 ~ 2025.06.30

담당	몬스터 AI 및 행동 제어 전반 (5인 중 몬스터 담당)
기반	DirectX 11 기반 자체 제작 엔진 (상용 엔진 미사용)
환경	C++, DirectX11 (HLSL 5.0, FX11), DirectXTK, Assimp, IMGUI, FMOD, PhysX
다루는 것	행동트리 기반 AI 구조 / 일정에 따른 구조 전환 판단 / 대량 사망 구간 프레임 스파이크 / 경로 이동과 애니메이션 분리 / 연출 중 스폰 제어
영상	youtu.be/ne0xWQ3QdaQ?t=1378



행동트리 기반 AI 구조

구조

BehaviorTree

루트 노드 실행, 진행 중 노드 유지, 리셋 플래그, JSON 저장/로드, Task/Decorator/Composite 팩토리 보유

BaseNode

행동트리 실행 →조건 선검사→노드 실행→조건 후검사→결과 반환(FAIL/ SUCCESS/CONTINUE/ABORT/NONE)

CompositeNode

- SEQUENCE
- SELECTOR
- PARALLEL

DecoratorNode

- Perform_ConditionCheck
- On_Notify
- ABORT_SELF / LOWER / BOTH

TaskNode

- Task_Wait
- Task_PlayAnimation
- Task_MoveTo
- Task_FinishWithResult

Decorator 구현 13종 - 조건 검사, 블랙보드 값 설정, 거리/각도, 쿨다운, 커스텀 실행 등

TimeLimit	CoolDown	FarFromTarget	CloseToTarget	DistanceToTarget
AngleToTarget	RotateToTarget	BlackBoard	LateBlackBoard	CustomRun
CustomLateRun	SetBlackBoardValue		SetBlackboardRandomValue	

BlackBoard

map<_string, VariantTypes> 변수, 타입 맵, 값 변경 델리게이트 - 노드끼리 직접 참조하지 않고 통신

에디터 - 데이터 ImGui 트리 편집 → 몬스터 행동 트리 저장/로드

설계 의도

몬스터마다 행동을 따로 구현하면 같은 행동이 중복되고, 조건이 늘수록 어떤 상황에서 무엇이 실행되는지 추적하기 어렵습니다.

구성

조건/흐름/행동/데이터를 노드 종류로 나눠 엔진 컴포넌트로 만들었습니다. 노드는 실행 전 조건 검사와 실행 후 재검사를 나눠 처리하고, 진행 중인 노드를 트리가 기억해 다음 프레임에 이어갑니다.

팀 접점

트리는 ImGui 에디터로 편집해 JSON으로 저장하므로, 몬스터 행동을 바꿀 때 코드를 다시 빌드하지 않습니다. 몬스터 속성은 구조체로 주입하고 이벤트는 델리게이트로 넘겨 외부에서 붙이도록 했습니다.

일정 안에 필요한 행동 패턴 수를 채우지 못하던 문제

문제 해결

행동트리로 추가할 때

행동 정의 → Task, Decorator 클래스 작성 → 에디터에서 노드 배치 /연결 → Blackboard 키 정의 → JSON 저장 /재로드

FSM으로 추가할 때

상태 추가 → 진입, 전이 조건 기술 → 애니메이션 지정

CHARACTERSTATE 11종 - EasyFSM 구현 9종

IDLE 대기	**WALK** 걷기	**RUN** 달리기	**ATTACK** 공격
TRANSITION 전이 연출	**GRAPPLE** 잡기, 합동기	**HIT** 피격	**GROGGY** 그로기
DEAD 사망	**LATE_DEAD** 지연 사망		

Soldier, Mendez, Novistador, Plaga, Saddler, Grapper ...

주황 = 대부분의 상태에서 전이 가능한 우선 상태

문제

5인, 2개월 일정에서 몬스터 타입과 행동 패턴을 계속 늘려야 했는데, 남은 기간 안에 필요한 수를 채우기 어려웠습니다.

원인

행동트리는 행동 하나를 넣을 때 노드 클래스 작성, 에디터 배치, 블랙보드 키 정의가 모두 필요해, 패턴 추가 속도가 일정 요구를 따라가지 못했습니다.

선택

구조적 확장성과 남은 기간 중 후자를 우선했습니다. 이미 만든 행동트리는 유지하고, 이후 추가되는 몬스터는 FSM으로 구현하는 절충안을 택했습니다.

구현

상태를 열거형으로 정의하고 전이를 직접 기술하는 EasyFSM을 만들어, 몬스터별로 상속해 진입, 전이, 종료만 채우도록 했습니다. 연속 애니메이션은 큐에 쌓아 순서대로 재생합니다.

검증

전환 이후의 주차별 패턴 추가 수를 기록해 목표 수량을 채웠는지 확인했습니다.

남은 한계

구조가 두 가지로 나뉘어 몬스터마다 코드를 읽는 방식이 달라졌습니다.

대량 사망 구간에서 프레임이 급격히 떨어지던 문제

문제 해결





문제

동시에 존재하는 몬스터가 많은 상황에서 광역 무기로 다수가 한 번에 사망하면, 그 순간 프레임이 급격히 떨어졌습니다.

원인

다수 몬스터가 동시에 제거, 재생성되면서 메모리 할당과 해제, 생성 비용이 한 프레임에 몰렸습니다. 사망 수가 늘수록 저하 폭도 함께 커졌습니다.

선택

동시 사망 수를 제한하면 광역 무기의 의도를 해치고, 사망 처리를 여러 프레임에 분산하면 연출이 늘어집니다. 비용 자체를 미리 치르는 풀링을 택했습니다.

구현

로딩 시점에 몬스터를 미리 생성해 비활성 상태로 보관하고, 사망 시 파괴 대신 풀에 반환한 뒤 초기값을 대입해 지정 좌표로 재배치했습니다.

검증

같은 조건에서 프레임 시간을 전후 비교해, 반복 생성, 삭제 비용을 줄이고 사망 순간의 런타임 스파이크가 완화되는 것을 확인했습니다.

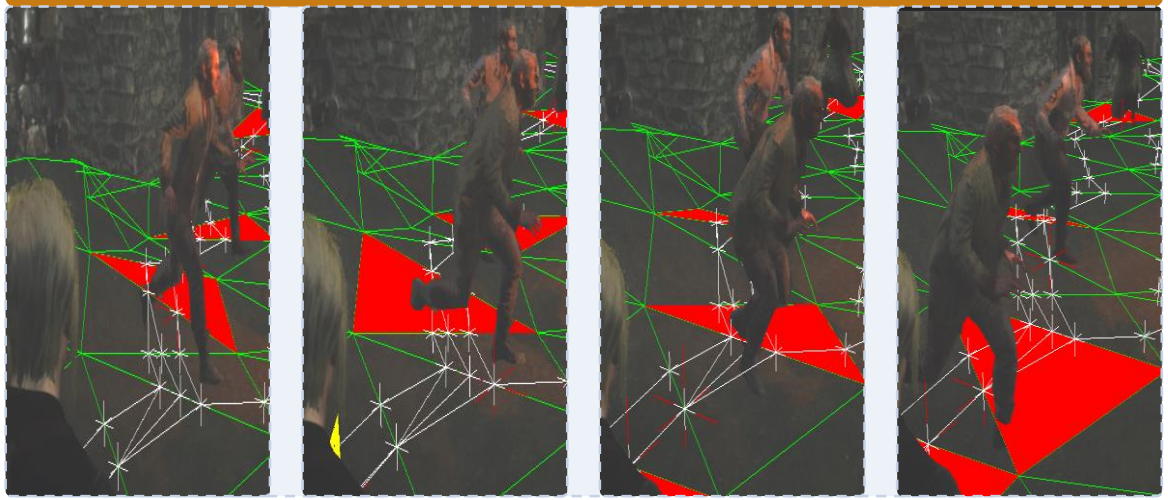
남은 한계

풀 크기를 고정값으로 잡아 예상보다 많이 필요할 때는 대응하지 못합니다. 사용량에 따라 늘리는 처리는 넣지 못했습니다.

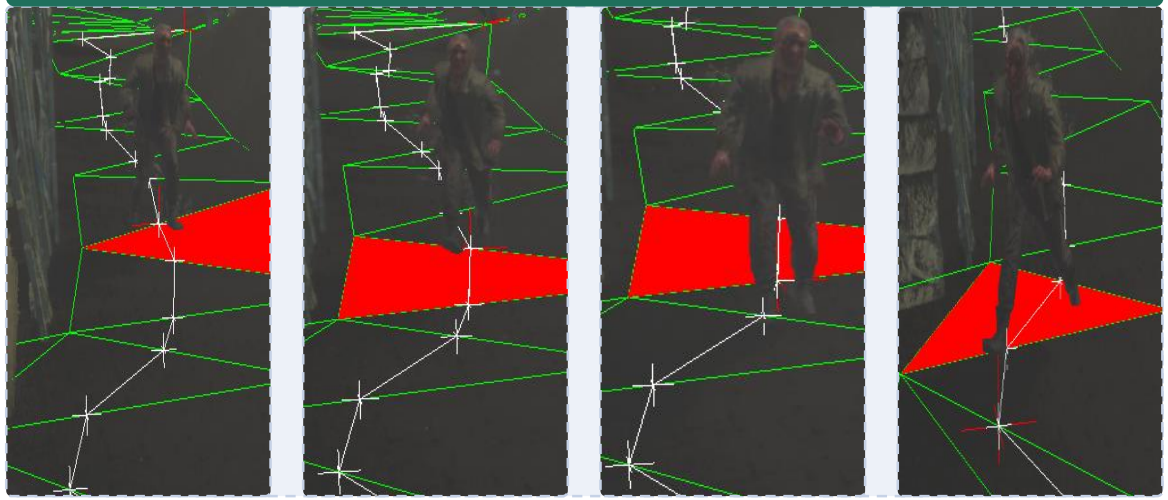
경로 이동과 루트 애니메이션이 서로 밀던 문제

문제 해결

Before



After



문제

내비게이션으로 이동 경로가 정해져도 루트 애니메이션이 적용되어 이동값이 애니메이션에 종속되면서, 추적 동작이 어색해졌습니다.

원인

이동 방향은 경로가 결정하고 이동 거리는 애니메이션이 결정해, 두 값이 서로 다른 기준으로 움직이고 있었습니다.

선택

루트 애니메이션을 끄면 이동 속도감이 사라지므로, 두 값을 모두 쓰되 역할을 나누는 쪽을 택했습니다.

구현

애니메이션에서는 이동 거리만 가져오고 방향은 경로의 다음 목적지 벡터에서 가져와 곱한 값으로 좌표를 갱신했습니다. 회전은 목적지 방향으로 서서히 보정했습니다.

검증

장애물을 우회해야 하는 구간에서 경로 이탈 없이 도착하는지, 이동 속도가 애니메이션과 어긋나지 않는지 반복 확인했습니다.

남은 한계

회전 보정 속도가 고정값이라 급격히 꺾이는 경로에서는 몬스터가 살짝 미끄러져 보입니다.

컷신 진행 중에도 몬스터가 등장하던 문제

문제 해결

서바이벌 이벤트





문제

시간 기반 몬스터 리스폰이 연출 상태와 무관하게 동작해, 컷신 진행 중에도 스폰 조건이 충족되어 몬스터가 등장했습니다.

원인

시간과 스폰 조건을 객체마다 따로 관리해 기준이 흩어져 있었고, "스폰 금지"라는 상태를 한 곳에서 표현할 수단이 없었습니다.

선택

각 객체에 컷신 여부를 물어보는 방식은 참조가 늘어나므로, 이벤트 전체가 참조하는 공유 기준을 하나 두는 쪽을 택했습니다.

구현

이벤트 진행용 공유 시간 기준을 두고, 그 값이 정해진 임계 이상일 때만 스폰을 허용하도록 조건을 통일했습니다. 이벤트 종료 시 동작은 타이머에 람다로 등록했습니다.

검증

컷신 구간을 반복 재생하며 의도치 않은 스폰이 발생하지 않는지 확인했습니다.

남은 한계

시간 임계값으로 판단하다 보니 컷신 길이가 바뀌면 값을 다시 맞춰야 합니다.

그 외 구현 — 몬스터 / 상호작용



일반 몬스터 클래스

속성 구조체를 생성 시 주입하고 타입별 모델, 충돌체, FSM을 불러 써, 타입 추가 비용을 줄였습니다.



보스 몬스터

공통 동작은 FSM에, 기믹 조건은 보스 클래스에 두고 컷신용 파이널 페이즈를 분리했습니다.



합동 애니메이션

공격, 합동기 타입을 담은 구조체로 상대의 피격 함수를 호출해 두 객체가 같은 연출을 재생합니다.

감사합니다

시스템 구조와 실행 흐름을 기반으로 문제를 분석하고, 설계를 통해 해결하는 게임 클라이언트 개발자입니다.

PROJECT 01	Unity 6 3D 액션 게임 (1인) - 저장소	github.com/CHGangRiO/p-11u-mg-ps
PROJECT 02	DirectX11 3D 액션 게임 (1인) - 플레이 영상	youtu.be/BVLWR92e69k
PROJECT 03	DirectX11 3D 액션 게임 (5인) - 플레이 영상	youtu.be/ne0xWQ3QdaQ?t=1378

김창홍 | 010-4720-0578 | 1138aiie@gmail.com